

TP do Aeroporto

Prof. Daniel Conrado

1 Descrição

O objetivo deste projeto é simular os padrões de aterrissagem e decolagem em um aeroporto utilizando as estruturas de dados que implementamos na disciplina.

```

      _ _ \ _ _
     '==/ \== '
    /-----/ \-----\
   /-----\
  / _ _ | _ _ | _ _ / . - - . \ _ _ | _ _ | _ _
 / _ _ | _ _ | _ _ ( > < ) _ _ | _ _ | _ _ \
   _ / ' - - ' \ _
  (/-----\ )

      |         | _ _ _ \         / ~ ~ ~ |
     _ : _ _ _ _ _ | / ' ( . . ) ' \ _ _ _ _ _ / | _ _ |
    < _ | ' ' ' ' ' ' \ _ _ ~ _ _ / _ _ _ | _ _ |
       : \ _ _ _ _ (=====, (*), - - \ _ _ | _ /
       |         \         / - - - '
           |         | (* ) /
           | _ _ _ _ /

```

Suponha um aeroporto que possui 3 pistas, numeradas 1, 2 e 3. Existem 4 (quatro) “prateleiras” de espera para aterrissagem, duas para cada uma das pistas 1 e 2. Aeronaves que se aproximam do aeroporto devem se integrar a uma das prateleiras (filas) de espera, sendo que estas filas devem procurar manter o mesmo tamanho. Assim que um avião entra em uma fila de aterrissagem, ele recebe um número de identificação ID e um outro número inteiro que indica o número de unidades de tempo que o avião pode permanecer na fila antes que ele tenha que descer (do contrário seu combustível termina e ele cai).

Existem também filas para decolagem, uma para cada pista. Os aviões que chegam nestas filas também recebem uma identificação ID. Estas filas também devem procurar manter o mesmo tamanho. A cada unidade de tempo, de 0 a 3 aeronaves podem chegar nas filas de decolagem e de 0 a 3 aeronaves podem chegar nas prateleiras.

A cada unidade de tempo, cada pista pode ser usada para um pouso ou uma decolagem. A pista 3 em geral só é usada para decolagens, a não ser que um dos aviões nas prateleiras fique sem combustível, quando então ela deve ser imediatamente usada para pouso. Se apenas uma aeronave está com falta de combustível, ela pousará na pista 3; se mais de um avião estiver nesta situação, as outras pistas poderão ser utilizadas (a cada unidade de tempo no máximo 3 aviões poderão estar nesta desagradável situação).

Utilize inteiros pares (ímpares) sucessivos para a ID dos aviões chegando nas filas de decolagem (aterrissagem). A cada unidade de tempo, assuma que os aviões entram nas filas antes que aterrissagem ou decolagens ocorram. Tente

projetar um algoritmo que não permita o crescimento excessivo das filas de aterrissagem ou decolagem. Coloque os aviões sempre no final das filas, que não devem ser reordenadas.

A saída do programa deverá indicar o que ocorre a cada unidade de tempo. Periodicamente imprima:

- o conteúdo de cada fila;
- o tempo médio de espera para decolagem;
- o tempo médio de espera para aterrissagem; e
- o número de aviões que aterrissam sem reserva de combustível.

Os itens *b* e *c* acima devem ser calculados para os aviões que já decolaram ou pousaram, respectivamente. A saída do programa deve ser auto-explicativa e fácil de entender.

A entrada poderia ser criada manualmente, mas o melhor é utilizar um gerador de números aleatórios. Para cada unidade de tempo, a entrada deve ter as seguintes informações:

- número de aviões (0-3) chegando nas filas de aterrissagem com respectivas reservas de combustível (de 1 a 20 em unidades de tempo);
- número de aviões (0-3) chegando nas filas de decolagem.

2 Como fazer

Primeiro, só comece a fazer depois de ter entendido bem o que deve ser feito. Se necessário, leia o enunciado novamente; não tenha pressa.

Comece identificando a *essência* do sistema: os algoritmos. É uma boa ideia escrevê-los em pseudocódigo primeiro. Assim, você não precisa ficar pensando em detalhes de uma linguagem de programação nem em como o usuário deve interagir com seu sistema, pois vai pensar apenas nos valores que entram e nos que saem.

Ao projetar os algoritmos, identifique as estruturas de dados adequadas, tentando, como exercício, priorizar a eficiência. Como dica, tente dividir a sua solução em algoritmos tais que cada um faça apenas uma tarefa coesa. Às vezes, um algoritmo recebe como entrada valores que já foram anteriormente processados por outro algoritmo. Tente testar seus algoritmos com testes de mesa.

Desenvolva o sistema em alguma linguagem de programação orientada a objetos (ou funcional, caso queira aventurar-se), como Java, Kotlin, C#, Python, Rust, Go (ou, se funcional, Scala, F#, Clojure, Haskell, Python, JavaScript, Elixir, OCaml). Faça uma pesquisa e escolha a IDE mais adequada para a linguagem desejada (nem sempre é o VSCode).

Para este TP, um sistema com saída apenas em texto é o suficiente. Opcionalmente (e se estiver com tempo), você pode experimentar fazer uma interface gráfica!

Converse com o professor e/ou com o monitor da disciplina periodicamente a respeito deste trabalho, no intuito de obter feedback e orientação.

3 O que entregar

Você deve entregar:

- o código-fonte (sem binários nem dependências, por gentileza);
- um relatório contendo:
 - A listagem dos testes executados.
 - Descrição sucinta (por exemplo, desenho), das estruturas de dados e as decisões tomadas relativas aos casos e detalhes de especificação que porventura estejam omissos no enunciado.
 - Estudo da complexidade do tempo de execução dos procedimentos implementados e dos programas como um todo (notação O).

Bons estudos! :-)